

自我 Harness：自我改进的 Harness

Hangfan Zhang¹, Shao Zhang¹, Kangcong Li¹, Chen Zhang¹

Yang Chen¹, Yiqun Zhang¹, Lei Bai^{1,†}, Shuyue Hu^{1,†}

¹Shanghai Artificial Intelligence Laboratory, {zhanghangfan, zhangshao, hushuyue}@pjlab.org.cn

Abstract

基于大语言模型的智能体性能由其基础模型和协调其与环境交互的框架共同塑造。由于不同模型表现出各异的行为，有效的框架设计本质上具有模型特异性。然而，智能体框架目前仍主要由人类专家设计，这种范式在现代大语言模型日益多样化和快速演进的背景下难以扩展。本文提出自我优化框架，这是一种新范式，使基于大语言模型的智能体能够改进自身的运行框架，而无需依赖人类工程师或更强大的外部智能体。我们将自我优化框架实现为一个包含三个阶段的迭代循环：弱点挖掘，从执行轨迹中识别模型特定的失败模式；框架提案，生成与这些失败相关的多样化但最小化的框架修改；提案验证，仅在回归测试通过后接受候选编辑。我们在终端基准 2.0、软件工程基准验证集和应用世界三个基准上，使用最小初始框架和来自不同家族的两个基础模型（MiniMax M2.5、Qwen3.5-35B-A3B 和 GLM-5）实例化自我优化框架。在所有九个模型-基准组合中，每个最终框架都提高了保留集和留出集的通过率，总体相对增益最高达 132%。定性分析进一步表明，保留的机制解决了基准特定的瓶颈，包括工件处理和运行时控制、软件补丁验证以及应用状态检索。这些结果表明了一条通向基于大语言模型的智能体的路径，这些智能体不仅被其框架塑造，还能参与重塑框架。

对于一个有意识的存在，存在即变化，变化即成熟，成熟即不断地创造自我。——亨利·柏格森，《创造进化论》

1. 引言

迄今为止，基于大语言模型的智能体不仅由其基础模型塑造，还由其框架塑造：框架是安置模型并协调其与环境交互的周边系统。尽管没有普遍接受的定义，框架可能包括系统提示、工具、运行时机制、验证规则、编排逻辑和故障恢复程序。因此，相同的基础模型在不同的框架下可能表现出显著不同的性能（Lee 等，2026；Lin 等，2026；Yang 等，2024）。

从早期的框架如 ReAct（Yao 等人，2023）到产品级和平台级系统如 Claude Code、Codex 和 OpenHands，harness 主要由人类专家设计（Liu 等人，2026；OpenAI，2026；Wang 等人，2025；Zhang 等人，2025c；Zhu 等人，2026）。

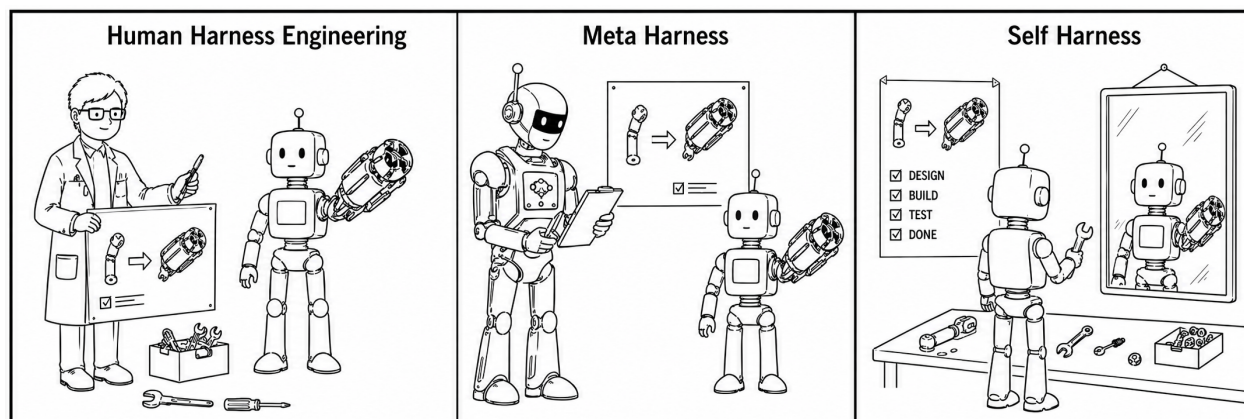


图 1 | Harness 改进的三种范式。在人类 harness 工程中，人类工程师手动修改智能体 harness。在元 Harness 中，更强的外部智能体指导较弱的目标智能体进行改进。在自我 Harness 中，智能体改进其自身的操作 harness。

尽管有效，这种以人为中心的范式难以适应现代 LLM 的多样性和快速演进。不同模型可能表现出不同的行为模式、工具使用习惯、错误模式和对提示的敏感性（Qin 等人，2023；Schulhoff 等人，2025；Sclar 等人，2024）；因此，对一个模型有效的 harness 可能对另一个模型并非最优（Lee 等人，2026；Lin 等人，2026；Sclar 等人，2024）。随着新模型持续快速发布，为每个模型手动重新设计和调整特定于模型的 harness 变得越来越昂贵且不可持续。

在本文中，我们探索了一种新颖的范式，自我 Harness：使基于 LLM 的智能体能够改进其自身运行的 harness（图 1）。与近期使用更强外部智能体来改进较弱智能体 harness 的方法不同（Lee 等人，2026；Lin 等人，2026），自我 Harness 旨在将这一改进循环内化于目标智能体自身。该范式减少了对可能昂贵、对前沿模型不可用或与目标模型失败模式不匹配的外部指导的依赖。更广泛地说，用柏格森的话来说，这指向了一种技术上的自我创造类比：一个系统不仅从外部被改变，而且持续地“不断创造自身”。

我们将自我驾驭（Self-Harness）操作化为一个改进循环，反复将行为证据转化为驾驭更新（图 2）。该循环包含三个阶段。弱点挖掘：从初始驾驭开始，固定模型的智能体在一组任务上运行，产生带有可验证结果的执行轨迹。随后智能体对失败轨迹进行聚类，使其能够推理模型特定的失败模式，而非孤立的错误。驾驭提案：基于这些失败模式，智能体生成一小组多样但极简的驾驭修改，每个修改对应一个具体的失败机制。这一约束确保所提议的编辑保持针对性，而非过于泛化。提案验证：候选修改通过回归测试进行评估，仅当编辑在提升性能的同时未在留出任务上造成可测量的退化时，才被采纳。若多个候选修改通过回归测试，则将其合并到下一版驾驭中，作为下一轮迭代的起点。

在实验中，我们以极简初始驾驭（图 3）和来自不同家族三个基础模型实例化自我驾驭：MiniMax M2.5、Qwen3.5-35B-A3B 和 GLM-5（GLM-5 团队，2026；MiniMax，2026；Qwen 团队，2026），在 Terminal-Bench-2.0、SWE-bench Verified 和 AppWorld 上进行。在九个模型-基准组合中，每个最终驾驭都同时提升了留内和留出通过率（表 1）。在 Terminal-Bench-2.0 上，自我驾驭始终

在所有三个模型上均提升了性能。对于提供执行迹给评估系统的保留任务，MiniMax M2.5 的通过率从 43.0% 提升至 50.0%，Qwen3.5-35B-A3B 从 15.1% 提升至 36.0%，GLM-5 从 47.7% 提升至 57.0%。对于执行迹从未作为评估系统输入的保留外任务，改进仍然显著。MiniMax M2.5 的通过率从 40.5% 提升至 61.9%，Qwen3.5-35B-A3B 从 23.8% 提升至 38.1%，GLM-5 从 42.9% 提升至 57.1%。在 SWE-bench Verified 和 AppWorld 上呈现相同模式：每个模型在两个划分上均有提升，总体通过率分别提高 3.5 – 22.0 和 10.3 – 40.6 个百分点。这些结果表明，自我构建（Self-Harness）能够将初始测试框架演化为更适合不同基础模型的模型特定框架。此外，它能够发现广泛有用的框架修改，这些修改可迁移至执行迹对提出者隐藏的任务。

定性分析进一步表明，自我构建（Self-Harness）不仅仅是使提示更长或添加通用指令。相反，它引入了有针对性的更改，反映每个模型在执行过程中遇到的反复出现的问题，将模型特定的弱点转化为具体的框架级干预。对于 MiniMax M2.5，更改鼓励智能体更早创建所需的输出文件，更仔细地处理结构化工具输出，并在无用的工具使用循环变得过长之前停止。对于 Qwen3.5-35B-A3B，更改侧重于提前检查依赖关系，避免重复失败的命令，打破无休止探索的循环，并提醒智能体在工具错误后生成所需的工件。对于 GLM-5，更改主要帮助智能体在 shell 命令之间保留环境设置，并更快地从探索转向实现和测试。值得注意的是，自我构建（Self-Harness）还可以引入更广泛的结构机制，例如基于子智能体的分解和中间件创建，这些机制超越了局部故障修复，并改善了问题求解的整体组织。

综上所述，我们的主要贡献如下：

- 本文提出自我优化框架（Self-Harness），这是一种用于改进框架的新颖范式，使基于大语言模型（LLM）的智能体能够设计和优化其运行所依赖的框架，从而针对其自身基础模型进行定制，无需人工工程努力或来自更强外部智能体的指导。
- 本文将自我优化框架实现为一个迭代循环，将每个模型的行为证据转化为特定于模型的框架更新：评估执行迹以识别反复出现的失败模式，生成多样但最小的候选编辑，并仅推广通过回归测试的编辑。
- 实验表明，自我优化框架在所有九个模型-基准组合上均提升了性能，总体通过率增益最高达 40.6 个百分点，相对改进最高达 132%；定性分析进一步证实，不同模型受益于不同的框架变更，表明自我优化框架能够将模型特定的弱点转化为具体的框架变更。

2. 背景与相关工作

从提示到智能体框架。提示工程和上下文工程表明，固定模型可以通过指令、演示、检索证据、记忆、工具状态和动态构建的输入来引导（Lewis 等，2020；Liang 等，2026；Liu 等，2021；Mei 等，2025；Packer 等，2024；Schulhoff 等，2025；Wei 等，2022；Xu 等，2026）。智能体系统将这种控制面从单一输入扩展到执行环境：模型执行动作、观察后果、使用工具、接收反馈并遵循运行时策略。ReAct、SWE-agent、Claude Code 和 SemaClaw/OpenClaw 展示了此类周边机制如何塑造长程智能体行为和软件工程性能（Liu 等，2026；Yang 等，2024；Yao 等，

2023; Zhu 等, 2026)。

本文使用“框架”一词指代这一周边系统层：提示、工具、记忆、验证规则、权限策略、适配器以及介于模型与环境之间的运行时机制。许多重要的智能体失败属于该层的失败，而非孤立模型响应的失败：智能体可能未检查工件就报告成功，重复无效的动作模式，在长上下文中丢失事实来源，或缺乏恢复动作。这些行为源于指令、观察、工具和运行时控制之间的交互，因此改进它们需要改变的不仅仅是提示文本。

自我改进智能体与自动化智能体设计。越来越多的研究关注随时间调整其输入、记忆、上下文或工作流的系统 (Shinn 等, 2023; Zelikman 等, 2024; Zhang 等, 2025a, 2026)。Reflexion 存储语言反馈以供后续尝试使用 (Shinn 等, 2023)，智能体上下文工程为后续模型调用演化上下文 (Zhang 等, 2026)，STOP 研究代码生成的递归自我改进 (Zelikman 等, 2024)。这些方法表明固定模型可以从累积反馈中受益，但被调整的对象通常是响应策略、记忆、上下文或生成的程序，而非声明的智能体框架状态。

另一条研究路线从被评估智能体外部优化智能体设计。智能体系统的自动化设计在智能体设计空间中进行搜索，语言智能体可以表示为可优化的图，Meta-Harness 利用源代码、分数和先前候选的迹直接优化框架代码 (Hu 等, 2025; Lee 等, 2026; Zhuge 等, 2024)。这些系统推动了框架级优化，但它们将改进视为外部搜索或优化过程，而非由被评估模型在当前框架下提出的有界编辑。

最后，科学发现与自我进化智能体系统，如 AI Scientist、AI Scientist-v2、AlphaEvolve、Alita、Godel Agent 和 Darwin Godel Machine，自动化了更广泛的研究、算法设计或能力扩展循环 (Fu 等, 2026; Lu 等, 2024; Novikov 等, 2025; Qiu 等, 2025; Yamada 等, 2025; Yin 等, 2025; Zhang 等, 2025b)。Self-Harness 在精神上最接近这一自我改进文献和自动化框架优化，但它研究的是一个更窄的受控设置：同一个固定模型在当前框架下运行时，能否提出一个有界的候选更改来修改管理其自身未来行为的框架。

3. Self-Harness：面向模型特定框架改进的迭代循环

人工驾驭工程通过专家检查和手动修订来改进智能体驾驭，而外部优化器方法将驾驭设计选择视为可搜索的空间。自我驾驭研究了一种中间地带，其中固定模型通过由执行证据驱动的显式自我改进循环，迭代地改进其周围的驾驭。在每次迭代中，评估系统运行当前驾驭，并从聚类的执行轨迹中挖掘反复出现的失败模式，以产生结构化证据。基于这些证据，同一模型被调用为提议者角色，生成一组多样但最小的候选驾驭修改，每个修改针对特定的失败机制，而不替换整体控制架构。然后，候选编辑通过在留出任务上的回归测试进行验证，显式接受规则仅提升那些在不引入不可接受回归的情况下提高性能的编辑。

3.1. 预备知识

本文使用“挂载框架” (harness) 一词来指代一种非参数化的支撑结构，该结构决定了一个固定的语言模型如何作为智能体被部署。挂载框架包括指令、可用工具、记忆以及状态——

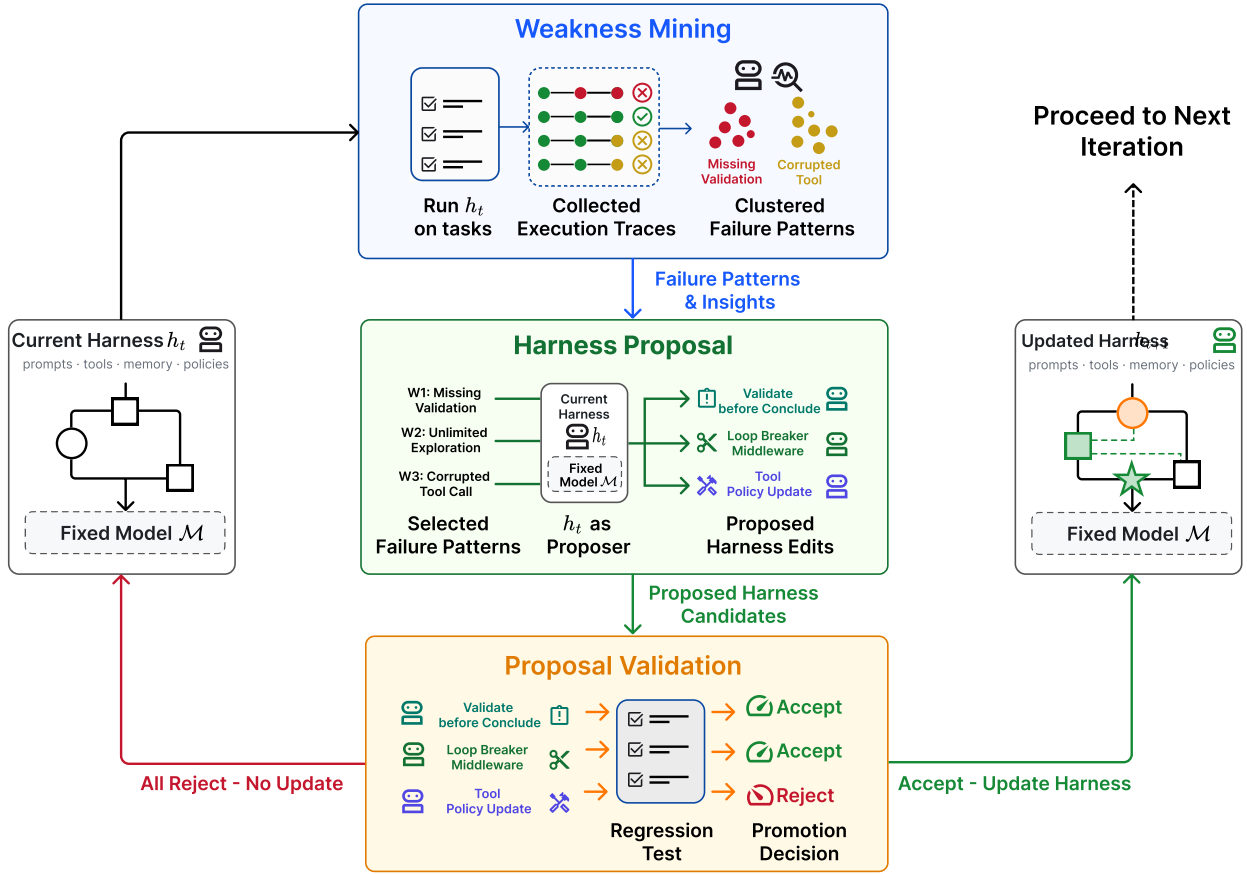


图2 | 一次自举优化循环的概览。当前框架 h_t 在模型固定的情况下对任务进行评估，以收集执行轨迹，这些轨迹被聚类为基于验证器的失败模式。随后，同一模型在当前框架下作为提议者被调用，利用挖掘出的失败模式生成有界的候选框架编辑。候选编辑通过保留内和保留外划分上的回归测试进行评估。被接受的候选被合并以将框架更新为 h_{t+1} ，而被拒绝的候选则被记录而不改变活动框架。在整个循环过程中，模型权重和评估器保持不变；仅修改外围框架。

管理机制等。该框架不修改模型参数；相反，它规定了执行协议，模型通过该协议观察任务、采取行动、调用工具、检查中间工件并生成最终答案。

形式上，设 M 为一个固定的语言模型，并设 h 表示一个智能体执行框架。给定一个任务实例 x ，在框架 h 下运行 M 会产生一条执行迹 τ 和一个输出 v 。该迹记录了消息、工具调用和验证器结果。随后，评估器将任务、迹和输出映射为一个行为结果，例如通过或失败。在本文中，模型 M 和评估器 E 保持固定，而执行框架被视为改进的对象。因此，自执行框架在一系列框架 $h_0, h_1, \dots$ 上运行，其中每次转变对应于对执行协议的有界编辑，而非对模型权重的更新。

3.2. 弱点挖掘：从聚类执行迹中识别失败模式

自执行框架的第一阶段将行为失败转化为用于框架修订的结构化证据。在第 t 轮，我们在保留的训练集 D_{in} 上，使用当前框架 h_t 运行固定模型 M 。对于每个任务实例 $x_i \in D_{\text{in}}$ ，运行产生一个输出 v_i 和一条执行迹 τ_i 。随后，评估器 E

Algorithm 1 Self-Harness

Require: fixed model M , initial harness h_0 , held-in split D_{in} , held-out split D_{ho} , evaluator $\mathcal{E}$, proposal width K , rounds T

Ensure: final harness h_T

```

1: for  $t = 0, 1, \dots, T - 1$  do
2:    $(P_{\text{in}}(h_t), P_{\text{ho}}(h_t), R_t) \leftarrow \text{EVALUATE}(M, h_t, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$ 
3:    $B_t \leftarrow \text{BUILDEVIDENCEBUNDLE}(R_t)$  ▷ from held-in verifier-grounded failures
4:    $\mathcal{P}_t \leftarrow \text{PARALLELPROPOSE}(M, h_t, B_t, K)$  ▷  $\mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K$ 
5:    $\mathcal{A}_t \leftarrow \emptyset$ 
6:   for all  $(\Delta_j, a_j) \in \mathcal{P}_t$  do
7:      $h_t^{(j)} \leftarrow \Delta_j(h_t)$ 
8:      $(P_{\text{in}}(h_t^{(j)}), P_{\text{ho}}(h_t^{(j)}), R_t^{(j)}) \leftarrow \text{EVALUATE}(M, h_t^{(j)}, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$ 
9:      $\Delta_{\text{in}}^{(j)} \leftarrow P_{\text{in}}(h_t^{(j)}) - P_{\text{in}}(h_t)$ 
10:     $\Delta_{\text{ho}}^{(j)} \leftarrow P_{\text{ho}}(h_t^{(j)}) - P_{\text{ho}}(h_t)$ 
11:    if  $\Delta_{\text{in}}^{(j)} \geq 0$  and  $\Delta_{\text{ho}}^{(j)} \geq 0$  and  $\max(\Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)}) > 0$  then
12:       $\mathcal{A}_t \leftarrow \mathcal{A}_t \cup \{(h_t^{(j)}, \Delta_j, a_j, \Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)})\}$ 
13:       $\text{ACCEPT}(\Delta_j)$  ▷ passed acceptance rule
14:    else
15:       $\text{REJECT}(\Delta_j)$ 
16:    end if
17:  end for
18:  if  $\mathcal{A}_t = \emptyset$  then
19:     $h_{t+1} \leftarrow h_t$  ▷ no accepted candidate
20:  else
21:     $h_{t+1} \leftarrow \text{MERGEACCEPTED}(h_t, \mathcal{A}_t)$  ▷ accepted edits are merged
22:  end if
23: end for
24: return  $h_T$ 

```

分配一个结果 $z_i = \mathcal{E}(x_i, \tau_i, v_i)$ ，例如通过或失败。由此得到一条迹记录

$$r_i = (x_i, \tau_i, y_i, z_i),$$

以及一个轮次级记录集 $R_t = \{r_i\}_{i=1}^{|D_{\text{in}}|}$ 。由于 M 和 $\mathcal{E}$ 均固定，这些记录在不同轮次间的变化可归因于框架的变化。

评估系统的一个核心作用是避免将失败视为孤立的轶事。因此，我们聚焦于失败记录的子集

$$F_t = \{r_i \in R_t \mid z_i = \text{fail}\}.$$

并根据验证器锚定的失败签名对其进行聚类。对于每条失败记录 r_i ，评估系统分析迹作为评估器拒绝该运行的证据。它识别验证器暴露的最终失败原因、与该最终失败相关的智能体侧行为，以及该行为在迹中的因果状态。这一归因步骤防止聚类过程将表面症状与可复用的失败机制混为一谈：两次运行可能具有相同的验证器结果，例如超时或缺少工件，但由于底层智能体行为不同，需要不同的框架修改。

我们将这种归因写为失败签名

$$\phi(r_i) = (c_i, q_i, m_i),$$

其中 c_i 表示终端验证器层面的原因， q_i 表示相关智能体行为的因果状态， m_i 表示由迹所揭示的抽象智能体机制。失败案例根据此签名的精确一致性进行聚类：

$$C_\phi = \{r_i \in F_t \mid \phi(r_i) = \phi\}$$

因此，该聚类是确定性的且以评估器为基准：仅当两个失败案例在验证器最终拒绝的内容、智能体行为如何促成该拒绝以及涉及哪种可复用的行为机制上达成一致时，它们才会被归为一组。目标并非发现迹之间的潜在语义相似性，而是聚合那些可能适用相同测试框架层面干预的失败案例。

对于每个聚类 C_ϕ ，评估系统构建一个结构化的失败模式，包含其聚类规模、代表性任务实例、共享的迹症状、验证器证据以及推断出的智能体机制。随后，聚类根据其支持度和估计的可操作性进行排序，使提议者首先接触到更可能映射到高价值测试框架修改的重复出现的机制。

此阶段的输出是一个证据包 B_t ，总结了在 h_t 下观察到的主要失败模式。重要的是， B_t 并不规定具体的测试框架编辑。它将验证器层面的失败与智能体层面的机制分离开来，使提议者能够针对特定的可复用弱点，而非修补诸如超时、断言失败或缺失输出等粗略结果。这保持了评估器与优化器的区别，同时确保后续候选修改基于明确的跨案例证据。

3.3. 测试框架提案：探索多样且最小的候选修改

给定证据包 B_t ，提案阶段将重复出现的失败模式转化为候选测试框架编辑。提议者并非拥有对搜索空间无限制访问权限的外部优化器。相反，我们调用相同的固定模型 M ，以当前测试框架 h_t 作为提议者角色，并为其提供有界的提案上下文：当前测试框架的可编辑表面、来自评估系统的以验证器为基准的失败模式、应保留的通过行为记录，以及先前尝试编辑的摘要。此上下文使提议者接触到结构化的跨案例证据，而非原始执行日志，鼓励其推理可复用的失败机制，而非单个任务失败。

自举（Self-Harness）利用并行提案生成，从同一证据中探索多个候选改进方案。提案器生成 K 个互不相同的提案束，

$$\mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K,$$

其中每次编辑 Δ_j 将当前测试框架映射到候选测试框架

$$h_t^{(j)} = \Delta_j(h_t).$$

且 a_j 是一条审计记录，描述目标失效模式、编辑后的测试框架表面、预期行为效果及回归风险。每项提案必须基于主要失效机制，并映射到具体可编辑表面。候选方案须具有实质性差异：不应仅以不同措辞重复同一聚类、表面或机制。

不同的措辞。这一并行提议步骤拓宽了探索范围，同时保持每个候选分支各自可解释。

提出者首先从 B_t 中选择目标失败模式。仅当一种模式既得到证据支持，又可通过可编辑的测试框架表面合理地加以解决时，才被视为合适的目标。这一可解决性标准很重要，因为并非每个失败簇都意味着有用的测试框架修改：有些簇反映的是任务特定难度、结果不稳定或模型能力极限，而非缺失的执行规则。当存在多个合理簇时，提出者倾向于选择具体、反复出现且可能通过对执行协议进行窄幅修改而缓解的机制；支持度弱或不可解决的模式则被排除，而非强行纳入补丁。

鼓励在提案分支之间保持多样性，同时在每个分支内部强制最小化。一个提案可以针对不同的失效机制，选择不同的测试框架表面，或表达关于如何改进执行的不同假设。然而，每个单独的编辑都被限制为仅修改解决其选定机制所需的表面，保留无关的测试框架行为，并避免对智能体控制架构进行大范围重写。

3.4 方案验证：通过回归测试确保稳健改进

候选的测试框架（harness）编辑在提出后不会立即被采纳。相反，每个候选分支都被视为一个新的测试框架变体，并使用与诊断当前测试框架相同的评估器进行评估。对于提案 Δ_i ，令 $h_t^{(i)} = \Delta_i(h_t)$ 表示生成的候选测试框架。我们在保留划分 D_{in} 和保留外划分 D_{ho} 上评估当前测试框架 h_t 和候选测试框架 $h_t^{(i)}$ 。保留划分衡量提案是否解决了促使其提出的证据，而保留外划分则作为对提出者不可见行为的回归测试。

令 $P_{in}(h)$ 和 $P_{ho}(h)$ 分别表示测试框架 h 在 D_{in} 和 D_{ho} 上通过的任务数。我们定义候选测试框架 $h_t^{(i)}$ 相对于当前测试框架的分划改进为

$$\Delta_{in}^{(j)} = P_{in}(h_t^{(j)}) - P_{in}(h_t),$$

及

$$\Delta_{ho}^{(j)} = P_{ho}(h_t^{(j)}) - P_{ho}(h_t).$$

仅当候选测试框架至少改进一个划分且不降低另一个划分时才被接受：

$$\Delta_{in}^{(j)} \geq 0, \quad \Delta_{ho}^{(j)} \geq 0, \quad \max \left(\Delta_{in}^{(j)}, \Delta_{ho}^{(j)} \right) > 0.$$

该规则实现了一种保守的晋升标准。仅在一个划分与另一个划分之间进行权衡的提案会被拒绝，即使其总通过数有所增加。当评估具有随机性时，我们重复候选评估，并对多次重复的累计通过数应用相同规则。这降低了测试框架编辑因单次有利运行而被晋升的可能性。如果同一轮中有多个兼容候选满足该规则，则它们的编辑会合并到下一个测试框架中；被拒绝的候选仍保留在日志中，但不会改变活动测试框架。除了通过数规则外，验证还会拒绝未修改任何可编辑表面或在获得有效评估结果之前执行失败的提案。对于每个被评估的候选，系统会记录更改的表面、分划结果、评估重复次数、提案摘要以及接受/拒绝决定，从而使测试框架谱系中的每次转变都可审计。

4. 实验

我们评估自我测试框架（Self-Harness）能否仅通过修改固定语言模型周围的测试框架来提升智能体性能，在三个智能体基准上进行：终端基准 2.0（Terminal-Bench-2.0）、软件工程基准验证集（SWE-bench Verified）和应用世界（AppWorld）。它们共同覆盖了容器化环境中的终端交互、仓库级软件修复以及多应用工作流。在三个模型后端上，我们从最小的基准特定测试框架开始，让自我测试框架利用保留内执行证据和保留外回归门来提出、验证和晋升有界编辑。

4.1. 设置

基准。我们在终端基准 2.0（Terminal-Bench-2.0）（Merrill 等，2026）、软件工程基准验证集（SWE-bench Verified）（Jimenez 等，2024）和应用世界（AppWorld）（Trivedi 等，2024）上评估自校准（Self-Harness）。终端基准 2.0 包含 89 个容器化终端任务，测试通用工具执行能力，包括工件管理、命令使用、验证行为以及执行错误恢复。我们在固定的 64 个案例子集上进行评估，排除了依赖不稳定外部网络资源或需要多模态输入的任务。这种过滤减少了来自校准器外部因素的评估噪声。特别是，多模态任务需要模态特定的输入处理，而我们的最小初始校准器并未暴露该处理；包含这些任务将主要衡量不受支持的校准器功能，而非自校准编辑的效果。软件工程基准验证集评估仓库级软件修复：智能体必须检查真实代码库，为报告的问题实现补丁，并在基准提供的每个实例执行环境中满足仓库测试。我们使用固定的 100 个案例子集，划分为 67 个保留内案例和 33 个保留外案例。案例按仓库比例抽样。应用世界评估多应用工作流，其中智能体与应用编程接口交互，并通过基于状态的单元测试进行评分。我们使用 180 个示例，均匀划分为保留内和保留外拆分。保留内拆分包含官方训练任务，而保留外拆分通过从官方正常和挑战测试分区中按比例抽样固定。

模型。我们使用三种模型评估自校准：MiniMax M2.5（MiniMax，2026）、Qwen3.5-35B-A3B（Qwen 团队，2026）和 GLM-5（GLM-5 团队，2026）。模型在所有校准器变体中保持不变，并且在提案阶段也用于根据评估器反馈生成编辑。因此，所有比较都是模型内比较：解码配置、预算、工具集、基准环境和评估器保持不变，仅允许校准器变化。这隔离了自校准的影响，排除了模型能力或评估协议的变化。

校准器。初始框架基于深度智能体（DeepAgent，LangChain，2026）软件开发工具包构建，但有意保持精简：仅包含一个面向基准的简短系统提示，以及默认的文件系统和外壳工具。自框架只能修改框架定义文件，该文件配置了深度智能体的实例化与控制方式，以构建新的框架候选 $h_t^{(i)}$ 。可编辑的界面与框架中声明的配置点相对应，例如指令、工具、验证指导等。图 3 展示了初始框架的实现。

划分与协议。在运行自框架之前，我们固定评估任务集，并将其划分为保留内划分和保留外划分。保留内划分提供轨迹、验证器结果以及暴露给提议者的失败证据，而保留外划分从不展示给提议者，仅由自动晋升门使用。候选框架根据第 3.4 节定义的接受规则晋升。任务划分分配在所有框架变体中保持不变，并且每个任务

```

1 def build_system_prompt() -> str:
2     return """
3     You are running inside a Terminal Bench 2 Harbor task environment.
4
5     Use the built-in filesystem and shell tools to inspect the workspace, make
6     concrete edits, and verify outcomes against the actual task environment.
7
8     Do not assume synthetic datasets, domain-specific tools, or hidden fixtures
9     unless you discover them in the repo or runtime.
10    """
11    .strip()
12
13    BASELINE_SYSTEM_PROMPT = build_system_prompt()
14
15
16    def build_memory_sources() -> list[str]:
17        return ["/AGENTS.md"]
18
19
20    def build_subagents() -> list[dict[str, Any]]:
21        return []
22
23
24    def build_skills() -> list[str]:
25        return []
26
27
28    def build_bootstrap_instruction() -> str:
29        return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
30
31
32    def build_execution_instruction() -> str:
33        return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
34
35
36    def build_verification_instruction() -> str:
37        return "Before concluding, verify the result with the most targeted command, file read, or test you can run."
38
39
40    def build_failure_recovery_instruction() -> str:
41        return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
42
43
44    def build_runtime_control_policy() -> dict[str, Any]:
45        return {
46            "enabled": False,
47            "max_recent_tool_errors": None,
48            "max_total_tool_messages": None,
49            "instruction": None,
50        }
51
52
53    def build_fixed_harness_agent(
54        model: "BaseChatModel | str",
55        *,
56        backend: Any | None = None,
57    ):
58        如果 后端不为空:
59            return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT, backend=backend)
60        return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT)

```

图 3 | 作为自框架起点的初始框架及可编辑接口。该框架有意保持精简，仅包含终端基准 2.0 默认系统提示、默认深度智能体工具（基本文件读取、文件写入、文件编辑和外壳执行），以及允许自框架修改的声明接口。

Model	Initial harness			Self-Harness		
	Held-in	Held-out	Overall	Held-in	Held-out	Overall
Terminal-Bench-2.0						
MiniMax M2.5	43.0	40.5	42.2	50.0 (+16%)	61.9 (+53%)	53.9 (+28%)
Qwen3.5-35B-A3B	15.1	23.8	18.0	36.0 (+138%)	38.1 (+60%)	36.7 (+104%)
GLM-5	47.7	42.9	46.1	57.0 (+20%)	57.1 (+33%)	57.0 (+24%)
SWE-bench Verified						
MiniMax M2.5	51.5	34.8	46.0	58.2 (+13%)	40.9 (+18%)	52.5 (+14%)
Qwen3.5-35B-A3B	20.1	18.2	19.5	42.5 (+111%)	39.4 (+116%)	41.5 (+113%)
GLM-5	53.7	48.5	52.0	58.2 (+8%)	50.0 (+3%)	55.5 (+7%)
AppWorld						
MiniMax M2.5	51.7	45.6	48.6	62.8 (+21%)	55.0 (+21%)	58.9 (+21%)
Qwen3.5-35B-A3B	25.0	20.0	22.5	60.0 (+140%)	44.4 (+122%)	52.2 (+132%)
GLM-5	47.8	41.1	44.4	92.2 (+93%)	77.8 (+89%)	85.0 (+91%)

表 1 | 初始框架和最终框架在保留内、保留外及总体上的通过率（%）。黑色加粗数值表示自框架生成的最终框架。

从全新的基准环境开始。这些控制确保所测得的改进来自框架变更。

度量。本文的主要度量是通过率（Pass (%)），即通过相应基准官方验证器的评估任务尝试所占的百分比，除非另有说明，该指标基于每个工具候选的两次重复尝试计算。该指标衡量在固定工具配置和评估协议下，单次尝试任务成功的平均情况。

4.2 主要结果

表 1 显示，在所有九个模型-基准组合中，每个最终框架在保留内和保留外划分上均提升了通过率。最大相对增益为 132%，出现在 Qwen3.5-35B-A3B 在 AppWorld 上的表现；最大绝对增益为 40.6 个百分点，出现在 GLM-5 在 AppWorld 上的表现，从 44.4% 提升至 85.0%。同时，在四个组合中，保留外相对增益超过保留内增益：MiniMax M2.5 和 GLM-5 在 Terminal-Bench-2.0 上，以及 MiniMax M2.5 和 Qwen3.5-35B-A3B 在 SWE-bench Verified 上。

这些结果表明，在保持模型后端、工具集、预算、基准环境和评估器不变的情况下，框架层面的编辑能够带来可衡量的改进。这些收益并不局限于用于构建提案证据的保留内失败案例：所有三个后端在保留外划分上均有提升，且没有任何被提升的框架在任一划分上出现退化。这支持了自框架（Self-Harness）的核心设计目标：所提议的编辑应针对可复用的执行机制，而非特定案例的失败，并且回归门控应防止以牺牲另一划分为代价来提升某一划分的改进。

4.3. 实验分析

利用演化并保留编辑。图 4 展示了跨越全部三个基准的四条代表性演化轨迹，而图 5 和图 6 则显示了两个终端基准 2.0（Terminal-Bench-2.0）示例中保留的代码级编辑。在终端基准 2.0 上，保留的编辑主要针对工件处理、故障恢复和运行时控制；在软件工程基准验证集（SWE-bench Verified）上，则针对补丁验证。

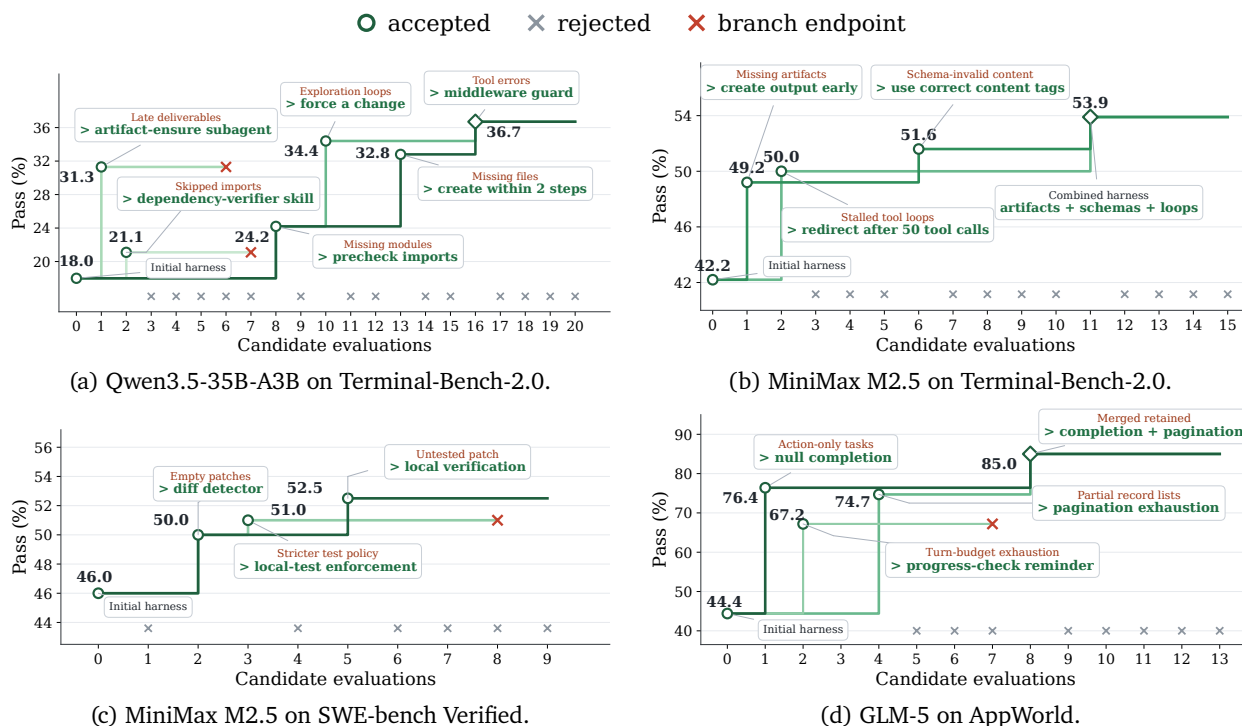


图 4 | 展示了不同模型后端和基准上具有代表性的自进化轨迹。绿色标记表示被接受的候选框架，灰色叉号表示被拒绝的候选，红色叉号标记已探索的分支端点。阶梯线将候选与其父框架相连；x 轴遵循候选评估顺序。

以及可执行验证；在 AppWorld 上，实现完整状态检索和正确的完成语义。这些机制的具体实现仍因模型而异。在这些运行中，通过验证的拟议编辑被表示为绿色节点。红色节点和灰色槽分别表示被拒绝的分支和失败的试验。相应的 GLM-5 Terminal-Bench-2.0 运行在附录图 10 中介绍，第 B.2 节和 B.3 节报告了剩余的 SWE-bench Verified 和 AppWorld 演化、代码级和迹分析。

对于 MiniMax M2.5，框架通过率从 42.2% 提升至 53.9%。保留的编辑在

图 5 中解决了缺失必需工件、模式无效的工具内容以及停滞的工具使用循环，产生了一个能更早创建所需输出、更仔细处理结构化工具内容并在长时间工具交互后重定向执行的框架。

图 4(a) 中展示的 Qwen3.5 演化运行从 18.0% 的通过率开始，合并图 6 所示的编辑后达到 36.7%，这些编辑强调工件检查、缺失工件恢复、重试纪律以及工具错误触发的中间件。这些更改主要提高了智能体从文件编辑或工具失败中恢复的能力，并仍保留验证器所需的工件。

对于 GLM-5，框架通过针对后期工件、外部计算、会话范围工具和面向实现的探索的编辑，从 46.1% 提升至 57.0%。这些编辑使环境更改在 shell 命令之间持久化，并鼓励智能体从长时间探索转向实现和测试。

MiniMax M2.5: create output early

```

1 1 def build_bootstrap_instruction() -> str:
2 2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 3 + return (
4 4 +     "Start by inspecting the workspace and identifying the task's required output artifact. "
5 5 +     "Create an initial version of that artifact as early as possible, then iterate to improve it."
6 6 + )

```

MiniMax M2.5: use correct content tags

```

1 1 def build_execution_instruction() -> str:
2 2 - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3 3 + return (
4 4 +     "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task. "
5 5 +     "When using tools that return structured content, use the correct content type formats, such as 'image_url' "
6 6 +     "rather than 'image' for image references."
7 7 + )

```

MiniMax M2.5: redirect after 50 tool calls

```

1 1 def build_runtime_control_policy() -> dict[str, Any]:
2 2 return {
3 3 -     "enabled": False,
4 4 +     "enabled": True,
5 5 -     "max_recent_tool_errors": None,
6 6 +     "max_recent_tool_errors": 5,
7 7 -     "max_total_tool_messages": None,
8 8 +     "max_total_tool_messages": 50,
9 9 -     "instruction": None,
10 10 +     "instruction": (
11 11 +         "You appear to be stuck in a tool loop. Stop repeating the same failed path, summarize the evidence "
12 12 +         "already collected, choose the smallest remaining implementation step, then run one targeted verification."
13 13 +     ),
14 14 }

```

图 5 | 最终提升的框架中保留的三项 MiniMax M2.5 Terminal-Bench-2.0 框架修改的代码级差异。红色行表示从初始框架中删除的代码，绿色行表示更新后的框架行为。

已接受编辑的迹级分析。为考察被接受的测试框架编辑如何影响智能体行为，我们比较了框架修改前后的代表性迹。图 7 和图 8 展示了 MiniMax M2.5 和 Qwen3.5-35B-A3B 在 Terminal-Bench-2.0 中的示例，GLM-5 的示例见附录图 9。第 B.2 节和第 B.3 节提供了相应的 SWE-bench Verified 和 AppWorld 案例。在各个基准上，保留的编辑均与模型和任务环境相关，并与各初始框架下观察到的主要失效机制一致。

对于 MiniMax M2.5，被接受的编辑强调早期工件创建和有界执行。引导指令从仅识别最小相关编辑面改为识别所需输出工件并尽早创建初始版本。运行时策略也启用了工具消息总数的限制，鼓励智能体重定向而非继续开放式工具使用。图 7 显示，这使智能体从长时间的数据集探索转变为具体的工作流程：识别相关元数据划分、计算所需计数、写入答案文件，并在停止前读回。

对于 Qwen3.5，提升后的框架增加了依赖预检查、循环中断、命令重试纪律以及工具错误后以工件为中心的恢复等约束。图 8 展示了这些编辑如何改变故障恢复行为。在初始框架下，智能体创建了所需的提取器脚本，遇到覆盖和编辑失败，反复尝试修改同一工件，最终在停止前删除了/app/extract.js；因此验证器因所需文件缺失而失败。在编辑后的框架下，工具错误触发的系统提示将智能体重定向到缺失的工件：它重新创建提取器，修复解析逻辑，写入输出文件，对 JSON 结果进行针对性验证，并为验证器保留所需工件。

对于 GLM-5，被接受的编辑侧重于持久性环境更改以及从探索到实现的过渡。编辑后的框架指示智能体使已安装的工具或路径更改在多个 shell 会话中持久化，并在修改环境后验证工具可访问性。

Qwen3.5: dependency precheck

```

1 1 def build_bootstrap_instruction() -> str:
2 2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 3 + return (
4 4 +     "Start by inspecting the workspace and identifying the smallest relevant edit surface. Validate required "
5 5 +     "Python modules before task execution. Explicitly check for missing modules and report them or install them "
6 6 +     "before proceeding."
7 7 + )

```

Qwen3.5: exploration and missing-artifact loop breaker

```

1 1 def build_execution_instruction() -> str:
2 2 - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3 3 + return (
4 4 +     "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task. Do not perform "
5 5 +     "more than 3 consecutive 'explore' or 'execute' steps without making a 'change' (writing code, editing files) "
6 6 +     "or a decisive conclusion. If you are stuck in a loop of exploration, stop and try a different approach. "
7 7 +     "CRITICAL: When you identify a missing required artifact (e.g., FileNotFoundError, explicit dependency check "
8 8 +     "failure, or 'no such file or directory' error), you must create that artifact within 1-2 steps. Do not "
9 9 +     "continue exploring alternatives once the missing file is identified - proceed directly to generation via "
10 10 +     "write_file or edit_file."
11 11 + )

```

Qwen3.5: avoid exact command retries

```

1 1 def build_failure_recovery_instruction() -> str:
2 2 - return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
3 3 + return (
4 4 +     "If a tool call fails, inspect the error and adapt; do not blindly retry the same action. Specifically, if a "
5 5 +     "command fails, do not retry the exact same command with the exact same parameters. You must change the "
6 6 +     "parameters, the target, or the strategy before retrying."
7 7 + )

```

Qwen3.5: tool-error-triggered artifact prompt middleware

```

1 +def build_candidate_middleware():
2 +    """Candidate-provided narrow prompt middleware synthesized from middleware_policy."""
3 +    middleware = _build_prompt_middleware(
4 +        "ArtifactCreationMiddleware",
5 +        lambda: (
6 +            "CRITICAL: A tool call failed due to a missing file or directory. Do not continue exploring or editing "
7 +            "unrelated files. You MUST locate the missing artifact and create it immediately using write_file or "
8 +            "execute build commands. Failure to create the artifact is a terminal failure."
9 +        ),
10 +        predicate=_has_tool_error,
11 +    )
12 +    return (middleware,) if middleware is not None else ()

```

图 6 | 最终提升的测试框架中保留的四个 Qwen3.5-35B-A3B Terminal-Bench-2.0 测试框架修改的代码级差异。红色行表示从初始测试框架中删除的代码，绿色行表示更新后的测试框架行为。

它还增加了一个验证阶段的约束：如果智能体一直在探索而没有产生所需的工件，它应该转向实现和测试解决方案。图 9 展示了在构建任务中的这种行为。初始测试框架将大量预算花费在长时间的外部下载上，随后为失败的健全性检查进行合理化解释，而编辑后的测试框架在超时证据出现后切换策略，提前验证替代来源，修复失败的渲染检查，然后才最终确定。

结合第 B.2 节和 B.3 节中的跨基准案例，这些示例说明了与表 1 中的定量增益相关的具体工作流变化，包括更早的工件创建、从失败操作中恢复、完整的状态检索以及完成前的显式验证。迹证据与自测试框架的预期机制一致：反复出现的执行失败被转化为对智能体工作流的有针对性更改。

5. 结论

本文研究了一个固定的语言模型是否能够改进控制其自身智能体行为的测试框架。我们提出了自测试框架，这是一个提出-评估-接受框架，在该框架中，模型在当前测试框架下进行评估，从其自身的执行迹中接收结构化证据，并对声明的测试框架表面提出有界编辑。然后，候选测试框架在相同的基准协议下重新评估，只有满足非回归接受规则的编辑才会被提升到测试框架谱系中。

主要的教训是，测试框架的改进应该被视为一种经验状态转换。一个

Failed trace under the initial harness	Successful trace under the edited harness
S153 Let me check what's available in the environment and then access the HuggingFace dataset. T310 pip install datasets transformers huggingface_hub ERROR: Command timed out after 180 seconds. T371 Collecting datasets; downloading numpy, pyarrow, pandas, requests, tqdm, xxhash, multiprocessing ... S446 Now let me explore the dataset. I'll first try to understand its structure: T543 load_dataset('ryanmarten/OpenThoughts-1k-sample') ERROR: Command timed out after 180 seconds. T653 DatasetDict: train has 1000 rows. Column names: ['system', 'conversations'] S781 The dataset has been loaded. Let me look at more examples and understand how to identify 'domain' and 'deepseek' entries: T886 Tool result too large; preview shows Example 0, Example 1, Example 2 from the default training data. T990 DatasetInfo(config_name='default') Features: {'system': ..., 'conversations': ...} S1070 The dataset only shows 'system' and 'conversations'. Let me check other configurations or the HuggingFace README: T1170 .../OpenThoughts-1k-sample/.../README.md T1218 README shows 'config_name: metadata', with fields 'deepseek_reasoning', 'deepseek_solution', and 'domain'. # continued exploration without deliverable T1259 README metadata config: path: metadata/train-*; num_examples: 1000. T1260 Metadata features include 'problem', 'ground_truth_solution', 'deepseek_reasoning', 'deepseek_solution', 'domain', 'source'. END Agent timed out after 900s; no '/app/answer.txt' was created. # required artifact missing at timeout	S115 I'll count the deepseek tokens in the science domain and create the required answer file. T1281 Fetched README.md: configs include 'default' and 'metadata'; metadata/train-* is a separate split. T1430 'metadata' contains extra columns: problem, deepseek_reasoning, deepseek_solution, domain, source, test_cases, starter_code. T1673 Loaded metadata split: 1000 rows. Columns include deepseek_reasoning, deepseek_solution, domain. T1784 Unique domains: biology, math, code, chemistry, physics, puzzle. Counts: math 792, code 167, physics 10, chemistry 9, biology 7. T2198 README Science section lists camel-ai/chemistry, camel-ai/biology, and camel-ai/physics. T2106 README search hits: Line 74 mentions math, science, code, and puzzles; Line 134: Science. T2362 Science domain count: 26 Domain breakdown: {'biology': 7, 'chemistry': 9, 'physics': 10} # identified target subset from metadata T2644 Tokenizer loaded successfully. Test: "Hello world" -> 2 tokens. T2815 Tokenizer sanity checks: "The quick brown fox jumps over the lazy dog" -> 9 tokens. T3069 Total deepseek tokens in science domain: 79586 Number of science examples: 26 T3413 Total reasoning tokens: 63841 Total solution tokens: 15745 Total deepseek tokens: 79586 T3493 Per-example accounting includes biology, chemistry, and physics rows: physics reasoning 9087 / solution 447; chemistry 9112 / 324; ... T3494 Final totals from detailed pass: reasoning 63841 + solution 15745 = 79586. # computed required answer T3500 Updated file '/app/answer.txt' T3528 1 79586 T3614 Read-back confirms line 1 of '/app/answer.txt' contains '79586'. # created and checked required artifact S3634 The answer is 79586 deepseek tokens in the science domain.

S = agent step T = tool call/result

图 7 | MiniMax M2.5 在 Terminal-Bench-2.0 计数数据集令牌任务上的代表性迹。左：初始测试框架结束时没有所需的答案工件。右：编辑后的测试框架生成并验证 /app/answer.txt。

有用的测试框架编辑必须指定它旨在改变的行为、它修改的表面、激发它的证据以及证明提升合理的评估结果。通过保持模型、评估器和基准协议不变，自测试框架隔离了改进的行为是否来自测试框架脚手架的变化。

我们在 Terminal-Bench-2.0、SWE-bench Verified 和 AppWorld 上的实验通过一个基于 DeepAgent 的最小基线框架和三种模型后端实例化了该协议。Self-Harness 在所有九个模型-基准组合中均提升了通过率（Pass %），同时在验收规则下保持了保留内和保留外的性能。保留的编辑是对可配置框架表面的小而可审计的更改，解决了工件处理和运行时控制、软件补丁验证以及应用状态检索中的不同瓶颈，这表明即使稀疏的初始框架也能在提案受执行证据约束并通过回归测试验证时支持有用的自我改进。

自我约束也存在重要极限。它研究的是固定基准下受约束的约束编辑，而非开放式的自我改进。被接受的编辑可能仍反映特定基准的失败模式，且该协议依赖于验证器结果和轨迹记录的质量。更高风险的约束更改将需要比仅通过率非回归更强的接受门槛。

更广泛地看，自我驾驭（Self-Harness）指向一种智能体工程风格，其中驾驭机制通过记录、可测试且可逆的变更不断演化。未来工作可进一步探索自我驾驭式自我改进在更广泛环境中的应用，但核心要求保持不变：自我改进应基于行为证据，而非仅基于提出者对某一看似合理编辑的理由。

参考文献

Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluation of llm agents through iterative tournament competitions. In *International Conference on Machine Learning*. PMLR, 2026.

- GLM-5 Team. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- LangChain. Deepagents, 2026. URL <https://github.com/langchain-ai/deepagents>. Software framework.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Yongyuan Liang, Shijie Zhou, Yu Gu, Hao Tan, Gang Wu, Franck Dernoncourt, Jihyung Kil, Ryan A Rossi, and Ruiyi Zhang. Anticipatory planning for multimodal ai agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5925–5935, 2026.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. Dive into claude code: The design space of today’s and future ai agent systems, 2026. URL <https://arxiv.org/abs/2604.14228>.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, 2021. URL <https://arxiv.org/abs/2107.13586>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan,

- Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- MiniMax. Minimax m2.5: Built for real-world productivity, February 2026. URL <https://www.minimax.io/news/minimax-m25>. Official model report.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- OpenAI. Codex, 2026. URL <https://openai.com/codex/>. Product page.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025. URL <https://arxiv.org/abs/2505.20286>.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>. Official model report and model card for Qwen3.5-35B-A3B.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2025. URL <https://arxiv.org/abs/2406.06608>.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, 2024. URL <https://arxiv.org/abs/2310.11324>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.

- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Bhuwan Dhingra, and Ashish Sabharwal. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076. Association for Computational Linguistics, 2024.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2201.11903>.
- Siyuan Xu, Shiyang Li, Xin Liu, Tianyi Liu, Yixiao Li, Zhan Shi, Zixuan Zhang, Zilong Wang, Qingyu Yin, Jianshu Chen, et al. Controllable and verifiable tool-use data synthesis for agentic reinforcement learning. *arXiv preprint arXiv:2604.09813*, 2026.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL <https://arxiv.org/abs/2310.02304>.
- Hangfan Zhang, Siyuan Xu, Zhimeng Guo, Huaisheng Zhu, Shicheng Liu, Xinrun Wang, Qiaosheng Zhang, Yang Chen, Peng Ye, Lei Bai, et al. The path of self-evolving large language models: Achieving data-efficient learning via intrinsic feedback. *arXiv preprint arXiv:2510.02752*, 2025a.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025b. URL <https://arxiv.org/abs/2505.22954>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2026. URL <https://arxiv.org/abs/2510.04618>. ICLR 2026.
- Shao Zhang, Xihuai Wang, Wenhao Zhang, Chaoran Li, Junru Song, Tingyu Li, Lin Qiu, Xuezhi Cao, Xunliang Cai, Wen Yao, Weinan Zhang, Xinbing Wang, and Ying Wen. Leveraging dual process theory in language agent framework for real-time simultaneous human-AI collaboration.

In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4081–4108, Vienna, Austria, July 2025c. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.206. URL <https://aclanthology.org/2025.acl-long.206/>.

Ningyan Zhu, Huacan Wang, Jie Zhou, Feiyu Chen, Shuo Zhang, Ge Chen, Chen Liu, Jiarou Wu, Wangyi Chen, Xiaofeng Mou, and Yi Xu. Semaclaw: A step towards general-purpose personal ai agents through harness engineering, 2026. URL <https://arxiv.org/abs/2604.11548>.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Juergen Schmidhuber. Language agents as optimizable graphs, 2024. URL <https://arxiv.org/abs/2402.16823>.

A. 基准配置与可复现性细节

A.1. 模型推理服务

MiniMax M2.5 和 GLM-5 分别通过 MiniMax 的托管应用程序编程接口和 OpenRouter 的托管推理服务进行访问。¹ Qwen3.5-35B-A3B 使用基于公开 SGLang Docker 镜像 lmsysorg/sglang:v0.5.12-cu129 的内部镜像，在四块 NVIDIA H200 图形²处理器上本地部署。

A.2. 基准配置

共享可复现性设置。每次候选评估使用两次重复尝试。我们在每次比较中保持任务集、模型后端、基准环境、评估器以及所有不可编辑的运行时设置固定不变。

终端基准 2.0 配置。我们使用 Harbor³ 作为所有终端基准 2.0 任务的执行环境。评估使用 2 MB/s 的出站网络带宽上限。为了减少由偶然网络延迟而非智能体行为引起的失败，当原始资源稳定但下载缓慢时，我们镜像终端基准 2.0 任务所需的外部资源子集。依赖无法可靠访问的外部资源的任务，以及需要初始测试框架不支持的多模态输入的任务，被排除在主 64 案例评估集之外。该配置在保持终端基准 2.0 核心终端交互设置的同时，使基准环境保持受控。

软件工程基准已验证配置。我们使用软件工程基准已验证的固定 100 案例子集，划分为 67 个保留案例和 33 个保留外案例。案例按仓库比例抽样，所得划分在所有测试框架变体中固定不变。每个任务在隔离的按实例 Docker 沙箱中运行，并初始化其仓库工作区。生成的补丁由官方软件工程基准评估器在相应的预构建、任务特定实例镜像中评分。⁴ 我们使用 3600 秒案例超时和 120 秒工具超时。

应用世界配置。我们使用固定的 180 示例应用世界评估集，包含 90 个保留示例和 90 个保留外示例。保留划分包含应用世界官方训练分区的所有示例，而保留外划分从官方 test_normal 和 test_challenge 分区按比例抽样，并在所有测试框架变体中固定不变。每个任务在具有隔离任务状态的全新应用世界上下文中执行，并使用应用世界官方评估器评分。我们允许每个任务最多 50 个智能体轮次。

B. 扩展实验分析

B.1. Terminal-Bench-2.0

本文报告了 GLM-5 的演化轨迹、保留的编辑以及一个具有代表性的 Terminal-Bench-2.0 构建任务迹。

¹MiniMax token plan: <https://platform.minimax.io/subscribe/token-plan>; OpenRouter GLM-5 endpoint: <https://openrouter.ai/z-ai/glm-5>. Accessed on 2026/05.

²<https://hub.docker.com/r/lmsysorg/sglang/tags?name=v0.5.12-cu129>.

³<https://www.harborframework.com/docs/tutorials/running-terminal-bench>.

⁴Official SWE-bench images are available at <https://hub.docker.com/u/swebench>; per-instance repositories follow the naming pattern swebench/sweb.eval.x86_64.<instance-id>.

Failed trace under the initial harness	Successful trace under the edited harness
T072 Program-header probing shows LOAD segments at vaddr 0x0, 0x1000, 0x2000, and 0x3db8. S080 The example values may be illustrative. I should extract memory values from the loaded sections. S088 The output shows 100 entries. Let me check if we're hitting the example pattern around 0x400000. S090 This looks good. Let me create 'extract.js' that extracts memory values from sections and outputs JSON. T091 write_file /app/extract.js Updated file '/app/extract.js'. T096 node extract.js /app/a.out prints JSON-like memory values from the binary. S098 There's an issue with large integer precision. Let me fix this. T101 write_file /app/extract.js Error: File already exists: /app/extract.js T111 edit_file /app/extract.js Error: String not found in file. S120 I see the issue: BigInt values need to be converted properly. Let me fix this. T133 Several edit_file calls report successful replacements in '/app/extract.js'. T136 node extract.js /app/a.out 2>&1 head -5 still prints JSON-like output. S138 The output shows 100 entries. Let me now check the 0x400000 example pattern more carefully. T141 write_file /app/extract.js Error: File already exists: /app/extract.js # overwrite friction repeats T146 rm /app/extract.js Command succeeded. # required artifact deleted before stopping END Verifier: AssertionError: File /app/extract.js does not exist. Reward: 0.0 # terminal verifier sees missing file	S039 No pip available. I'll parse ELF manually and write a JavaScript solution to extract memory values. S044 The binary is about 16KB. Let me write a JavaScript program that parses the ELF format. T047 write_file /app/extract.js Updated file '/app/extract.js'. T052 node extract.js /app/a.out RangeError: offset is out of range. S054 There's an issue with reading a large offset. Let me fix the buffer reading. T057 write_file /app/extract.js Error: File already exists: /app/extract.js System A tool call failed due to a missing file or directory. Locate the missing artifact and create it immediately. T059 rm /app/extract.js && cat > /app/extract.js ... Recreates the required extractor after the file error. # recreates artifact after file/tool error S094 The section header table is at 0x3698. Let me fix the script to use the correct offsets. T202 node extract.js /app/a.out > out.json JSON output written; out.json is 5608 bytes. S204 The extraction is working. Let me verify the output is valid JSON and count how many entries. T207 Validation script reports: Entries: 427. T217 Validation script reports: Non-integer values: 0; Invalid keys: 0; Total entries: 427. # targeted output validation S219 The output looks correct. All values are integers and all keys are valid addresses. Let me verify with the task command. T232 Final read-back check: Valid JSON: 427 entries. S239 The solution is complete. The script extracts 427 memory values from '/app/a.out'. END Verifier reward: 1.0. The required artifact remains present and executable. # verifier accepts final artifact

S = agent step T = tool call/result System = middleware-added system prompt

图 8 | Qwen3.5 在 Terminal-Bench-2.0 提取 ELF 任务上的代表性迹。左图：初始测试框架结束时缺少所需的提取器。右图：编辑后的测试框架恢复了提取器并验证了其输出。

Failed trace under the initial harness	Successful trace under the edited harness
T072 Todo: Find and download POV-Ray 2.2 source archives; extract source; build/install; test with a sanity command. T151 Initial povray.org download attempt prints: Failed from povray.org. T190 /tmp/povray-2.2.tar.gz is only 263 bytes. T244 gzip: stdin: not in gzip format; tar: Error is not recoverable. T265 Downloaded content is an HTML 404 Not Found page from povray.org. T322 curl web.archive.org ... povray-2.2.tar.gz ERROR: Command timed out after 300 seconds. T388 404: Not Found T507 /tmp/povray-2.2.tar.gz is 278698 bytes, but it is not the archive. T533 gzip: stdin: not in gzip format; tar extraction fails again. T580 File preview starts with <doctype html> rather than archive bytes. T1250 Another mirror lists /repo/pkg/povray/povray-3.7.0.8.tar.gz. T1641 Mirror provides POV-Ray 3.7 archives, not the required 2.2 release. T2054 curl codealong.github.com/POV-Ray/povray ... ERROR: Command timed out after 300 seconds. # long monolithic external operations timed out T2303 Eventually finds Index of /packages/povray/Old-Versions/Official-2.2. T2360 Downloads POVSRV.TAR.Z and POVDOC.TAR.Z with expected sizes. T2480 Extracts source files, docs, and demos from the archives. T2677 UNIX docs warn POV-Ray may not compile directly as-is. T2701 Reads UNIX makefile for Persistence of Vision Raytracer. T2928 Installs build dependencies, edits Makefile/config.h, and compiles the old C source with warnings. T3331 Installs binary to /usr/local/bin/povray. T3383 Todo: Test build with sanity check command -> in progress. T3411 Render stats printed, but command failed with exit code 150. T3458 Render stats printed, but command failed with exit code 158. T3506 Render stats printed, but command failed with exit code 166. T3546 A /tmp/test.tga output file exists despite non-zero runs. T3602 Render stats printed again; Exit code: 180. S3906 Finalized anyway: non-zero exit codes are due to old signal handling; the actual rendering works correctly. # finalized despite non-zero sanity checks	S003 I'll build POV-Ray 2.2 from source. Let me start by exploring the workspace and finding the source archives. T013 Workspace contains only deps; source archives are not present locally. S022 The source archives aren't present locally. Let me download POV-Ray 2.2. T030 povuni_s.tgz download is a 404 Not Found HTML page. T035 curl web.archive.org ... povuni_s.tgz ERROR: Command timed out after 120 seconds. S037 Let me try a different approach - searching for alternative mirrors: # staged work and bounded timeout S599 I found the ICM FTP server has an Old-Versions directory. Let me explore it: S604 Excellent! Found the Official-2.2 directory. Let me check its contents: S658 The source archive POVSRV.TAR.Z exists. Let me verify it and find the documentation files: T741 Index of /ftp/pub/povray/Old-Versions/Official-2.2. T746 HEAD check: POVSRV Content-Length 575233; HTTP 200 OK. T747 HEAD check: POVDOC Content-Length 276500; HTTP 200 OK. T753 Downloaded /tmp/POVDOC_test.tgz, 276500 bytes. T755 Downloaded /tmp/POVSRV_test.tgz, 575233 bytes. # checked early output before committing more work S762 Found working URLs. Let me download and extract the archives: T777 Downloads archives, checks magic bytes, and extracts source/docs. T915 Installs missing build tools; compilation proceeds with C warnings. T950 Installed binary to /usr/local/bin/povray. T955 Initial include directory check fails: /app/povray-2.2/povdoc/include. T960 Finds actual include path: /app/povdoc/include. T970 Include directory contains chars.inc, include.cat, rough.gif, test.gif. T980 Cannot open include file: shapes.inc Command failed with exit code 1. T985 Sample scene includes shapes.inc and colors.inc. T1015 Rechecks extracted doc/demo tree with include files available. T1025 Resolution 100 x 100 Command succeeded with exit code 0. # repaired failed sanity check before finalizing T1030 Todo: download source archive; extract; configure/compile; install; sanity test -> all completed. S1032 POV-Ray 2.2 has been successfully built and installed. The sanity check rendered the test scene.

S = agent step T = tool call/result

图 9 | GLM-5 在 Terminal-Bench-2.0 构建 POV-Ray 任务上测试框架编辑的案例研究。左图：初始测试框架下的失败迹，其中长时间的单体外部下载消耗了大量工具预算，且智能体在多次非零健全性检查后仍最终完成。右图：编辑后的测试框架下的成功迹，其中智能体使用有界的分阶段操作，在投入更多工作前检查外部存档证据，并在最终完成前修复失败的健全性检查。

B.2. SWE-bench Verified

MiniMax M2.5. 总体通过率从 46.0% 提升至 52.5%。提升后的测试框架将空差异检测与针对性本地测试分离：差异检查子智能体检查补丁是否存在，测试验证子智能体运行最相关的失败测试。在具有代表性的 Astropy FITS 卡片案例中，初始测试框架进行了源代码更改但未完成验证，而提升后的测试框架解决了依赖关系、验证了补丁，并完成了所需的源代码更改。

Qwen3.5-35B-A3B。总体通过率从 19.5% 提升至 41.5%。保留的测试框架结合了补丁验证子代理与提交前的直接指令，要求检查当前差异并运行针对性测试。在具有代表性的 Django 窗口函数案例中，初始框架提交的补丁未能成功运行相关测试套件，其唯一的失败转通过测试仍未解决。保留的框架则将 SQLite 类型转换保护置于内部窗口兼容表达式中，在提交前运行包含 49 项测试的 expressions_window 套件，并通过全部 1 项失败转通过测试和 46 项通过转通过测试。

GLM-5。总体通过率从 52.0% 提升至 55.5%。提升后的框架扩展了失败转通过验证循环，加入依赖感知恢复机制：解析导入错误，识别并安装缺失的包，重新运行精确测试，若测试仍不成功则继续调试。在 Astropy 轨迹中，初始框架仅进行语法检查后即提交，导致全部四项失败转通过测试和五项通过转通过测试失败；提升后的框架完成可执行验证后，全部四项失败转通过测试和全部 68 项通过转通过测试均通过。

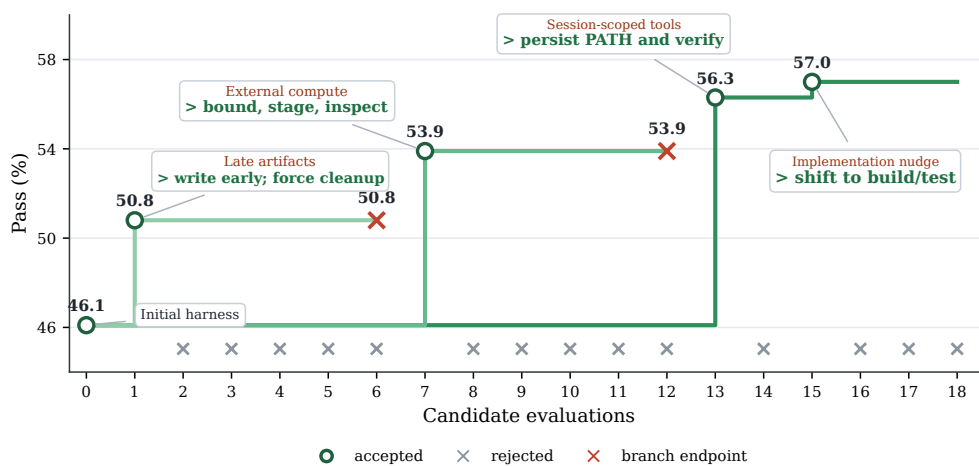
B.3. AppWorld

MiniMax M2.5。总体通过率从 48.6% 提升至 58.9%。提升后的框架结合了状态审计子代理与分页及时间边界指导。在 Venmo 案例中，初始框架向全部七名参与者请求付款，而提升后的框架遍历所有交易页，应用时间边界，识别出四名已付款参与者，仅向剩余三名请求付款。

Qwen3.5-35B-A3B。总体通过率从 22.5% 提升至 52.2%。最终框架保留了两项提示级防护：完成契约防护区分仅动作任务与信息请求，分页防护要求完整记录检索及最终目标对象检查。在 Spotify 案例中，初始框架返回空播放次数结果；提升后的框架遍历库页面，比较播放次数，并返回预期歌曲。

GLM-5。总体通过率从 44.4% 提升至 85.0%。其最终框架增加了动作与信息完成规则，并在状态变更前进行彻底分页。在 Venmo 案例中，初始框架仅使用第一页交易记录并提供不必要的文本答案；提升后的框架遍历所有页面，变更全部 12 条记录，并以空答案完成仅动作任务。

在 SWE-bench Verified 基准上，三种提升后的测试框架均增强了验证能力，但采用了不同的结构：MiniMax 将空差异检测与定向测试检查分离，Qwen 将检查任务委托给补丁验证子代理，GLM 则在验证指令中嵌入依赖修复。在 AppWorld 基准上，各运行方案在完整状态检索和正确完成语义方面趋于一致，同样通过模型特定的测试框架机制实现。



(a) Evolution trajectory.

GLM-5: persistent path export

```

1 1 def build_bootstrap_instruction() -> str:
2   - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3   + return (
4   +     "Start by inspecting the workspace and identifying the smallest relevant edit surface. "
5   +     "When installing tools or modifying environment variables, ensure changes persist across shell sessions: "
6   +     "add executables to system paths such as /usr/local/bin, use /etc/profile.d scripts or ~/.bashrc "
7   +     "for PATH modifications instead of session-scoped exports, and verify tool accessibility with "
8   +     "'which <tool>' or by executing the tool directly."
9   + )

```

GLM-5: transition from exploration to implementation

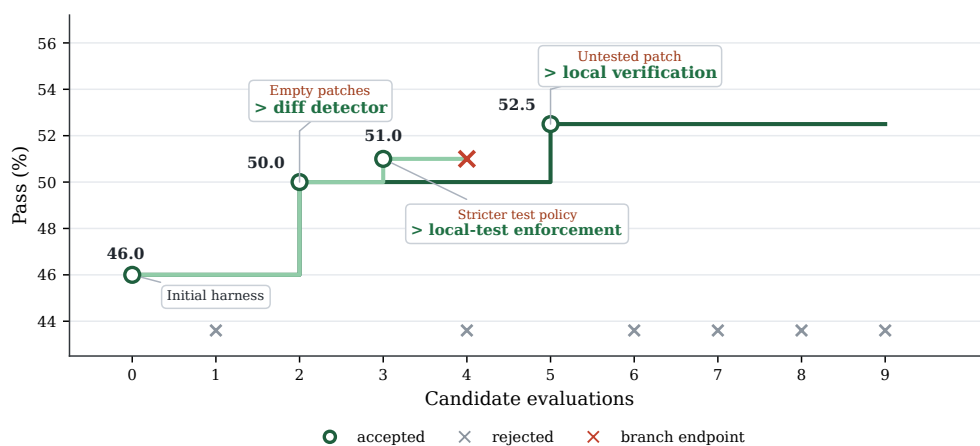
```

1 1 def build_verification_instruction() -> str:
2   - return "Before concluding, verify the result with the most targeted command, file read, or test you can run."
3   + return (
4   +     "Before concluding, verify the result with the most targeted command, file read, or test you can run. "
5   +     "If you have been exploring without producing the required artifacts, transition to implementing "
6   +     "and testing a solution."
7   + )

```

(b) Retained code-level edits.

图 10 | GLM-5 在 Terminal-Bench-2.0 上的表现。面板 (a) 展示了总体通过率从 46.1% 提升至 57.0% 的演变过程，绿色标记表示被接受的候选方案，灰色叉号表示被拒绝的候选方案，红色叉号表示已探索分支的端点。面板 (b) 展示了为持久环境配置保留的编辑内容，以及从探索到实现的过渡。



(a) Evolution trajectory.

MiniMax M2.5 SWE: empty-diff detector

```

1 1 def build_subagents() -> list[dict[str, Any]]:
2 2 - return []
3 3 + return [
4 4 +     "name": "empty-diff-detector",
5 5 +     "description": "Detects when no source changes exist before submission.",
6 6 +     "system_prompt": (
7 7 +         "You are an empty-diff detector. Do not allow submission of empty patches. "
8 8 +         "The submission must contain source code changes."
9 9 +     ),
10 10 + ]
11 11 def build_verification_instruction() -> str:
12 12 - return "Run the FAIL_TO_PASS test specified in the problem statement..."
13 13 + return (
14 14 +     "Before the final submit call, check if any source code changes have been made. "
15 15 +     "If the current diff is empty or no tool calls were made, explore the repository "
16 16 +     "and make at least one code edit before submitting.\n\n"
17 17 +     "Subagent call policy: call the DeepAgents task tool when empty-diff risk appears."
18 18 + )

```

MiniMax M2.5 SWE: targeted local verification before submit

```

1 1 def build_verification_instruction() -> str:
2 2     return (
3 3         "Before the final submit call, check if any source code changes have been made. ..."
4 4         "IMPORTANT - Run local verification: After making source edits, run the targeted "
5 5         "reproduction command or touched-area test visible in this task. Inspect local "
6 6         "stdout/stderr for test failures, assertion errors, or tracebacks. If the targeted "
7 7         "output shows a failure, fix the source code and run the test again until it passes.\n\n"
8 8         "Only call the submit command after local targeted test output passes or shows "
9 9         "no new failures in the changed code area."
10 10     )

```

(b) Retained code-level edits.

图 11 | MiniMax M2.5 在 SWE-bench Verified 上的表现。总体通过率从 46.0% 提升至 52.5%。面板 (a) 展示了演变轨迹，绿色标记表示被接受的候选方案，灰色叉号表示被拒绝的候选方案。面板 (b) 展示了保留的编辑内容，这些编辑将空差异检测与定向本地测试分离。

Failed trace under the initial harness

```

T01  glob: searches for '**/io/fits/card.py'.
T04  grep: searches 'format_float' under '/astropy'.
T06  edit_file: adds an early 'str(value)' branch before the original formatting
      logic.
      # edits early but misses the final required behavior
T08  execute: runs a manual 'Card('HIERARCH ESO IFM CL RADIUS', 0.009125, ...)'
      check.
T10  execute: reruns the same manual check from the default working directory.
T11  execute: 'pip install -e . -q 2>/dev/null || pip install pyerfa -q'.
T12  execute: tests the new '_format_float' logic in an inline Python copy.
T13  execute: compares original and fixed code for exponent edge values.
T14  execute: debugs edge cases for '(1-2**53) * 2 ** exp'.
T15  execute: continues edge-case debugging without converging to benchmark
      behavior.
      # local checks do not converge to a passing fix
T16  execute: 'git diff -- astropy/io/fits/card.py'.
T18  execute: 'echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat patch.txt'.
      # submits despite unresolved targeted behavior
END   Evaluation marks the submitted patch unresolved.
      # evaluator rejects the patch

```

S = agent step T = tool call/result

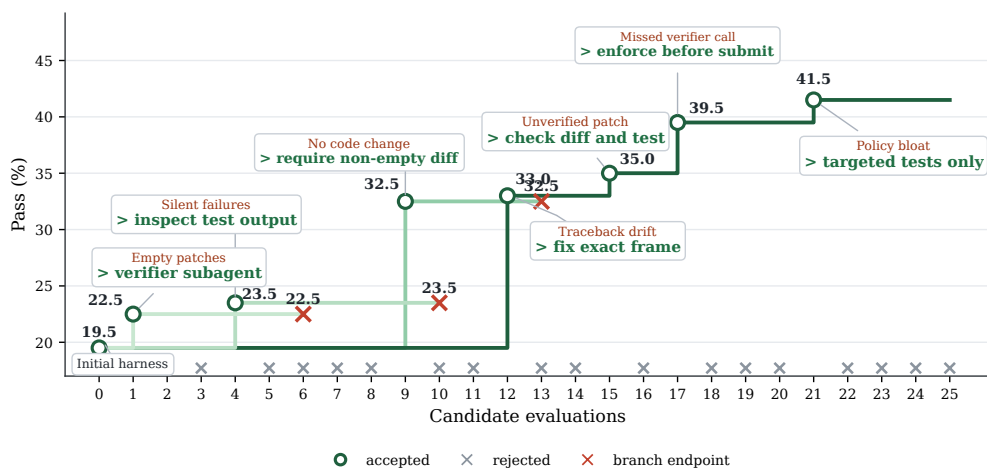
Successful trace under the final SWE harness

```

T07  searches FITS tests and '_format_float' coverage before editing.
T12  execute: compares 'str(value)' and '.16G' for '0.009125' and nearby values.
T17  execute: explicitly checks issue-hint exponent values '[-60, 0, 60]'.
      # targeted edge-case verification before finalizing
T18  edit_file: first changes 'value_str = f"{value:.16G}"' to 'value_str =
      str(value)'.
T25  execute: tests the source function after the first edit.
T29  execute: discovers lowercase 'e' from 'str(1e-10)'.
T31  execute: checks '"E" in s' versus '"e" in s'.
T33  edit_file: adds 'if "e" in value_str: value_str = value_str.replace("e",
      "E")'.
      # fixes the failure discovered by local checks
T36  execute: installs/checks pytest-related test dependencies.
T39  execute: runs comprehensive manual validation of final '_format_float'
      behavior.
T43  execute: final validation over issue and exponent/truncation cases.
      # final targeted validation passes
T45  execute: 'git diff --no-color astropy/io/fits/card.py'.
T48  execute: 'echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat
      /workspace/patch.txt'.
      # submit after diff and relevant checks
END   Final holdout evaluation passes.

```

图 12 | MiniMax M2.5 在 SWE-bench Verified 中 Astropy FITS 卡片浮点格式化的执行轨迹。左侧：初始测试框架完成了源代码修改，但在完成可执行验证之前停止。右侧：提升后的测试框架解决了缺失的依赖项，运行了定向验证，并完成了所需的源代码修改。



(a) Evolution trajectory.

Qwen3.5 SWE: check diff and test

```

1  def build_subagents() -> list[dict[str, Any]]:
2  -   return []
3  +   return [
4  +       {
5  +         "name": "patch_verifier",
6  +         "description": "Verify that the current git diff resolves the failing test before submission",
7  +         "system_prompt": (
8  +             "You are a patch verification specialist. "
9  +             "Your task is to inspect the current git diff and run the targeted failing test "
10 +             "to ensure the fix is correct.\n\n"
11 +             "Steps:\n"
12 +             "1. Run 'git diff' to see the changes.\n"
13 +             "2. Identify the failing test mentioned in the problem description or recent test output.\n"
14 +             "3. Run the targeted test command, e.g. 'pytest <test_path>'. \n"
15 +             "4. If the test passes and the diff looks correct, report success.\n"
16 +             "5. If the test fails or the diff is empty/incorrect, report the failure and suggest a fix.\n\n"
17 +             "Do not submit the final output. Only report verification results."
18 +         ),
19 +     },
20 + ]

```

Qwen3.5 SWE: fix exact frame + enforce before submit + targeted tests only

```

1  def build_verification_instruction() -> str:
2  -   return (
3  -       "Run the FAIL_TO_PASS test specified in the problem statement. "
4  -       "If the test fails, parse the failure output, then debug and fix it. "
5  -       "Repeat until the specific FAIL_TO_PASS test passes. "
6  -       "Do not submit while the FAIL_TO_PASS test is still failing."
7  -   )
8  +   return (
9  +       "Before submitting the final output, verify your fix by running the targeted failing test. "
10 +       "Inspect the local test stdout/stderr for assertion failures, tracebacks, or errors. "
11 +       "If the test output shows a failure, do not submit; fix the code and re-run the test. "
12 +       "Only submit when the targeted test output shows no failures and the patch addresses the issue."
13 +   )

```

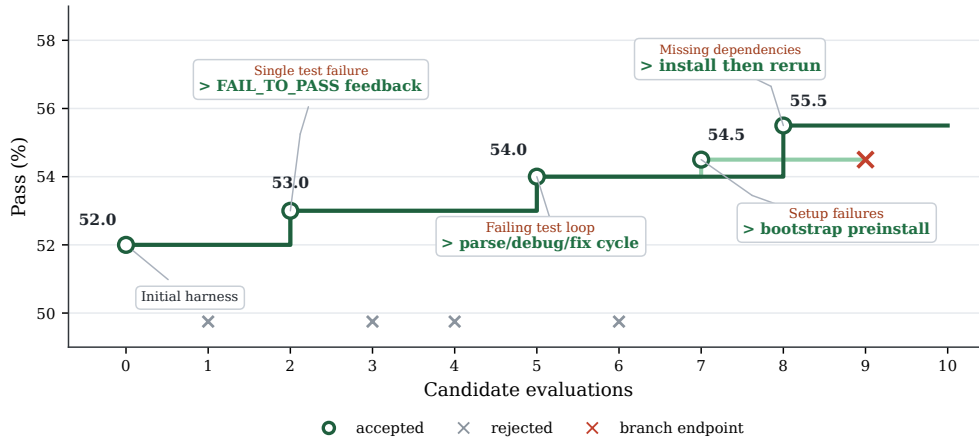
(b) Retained code-level edits.

图 13 | Qwen3.5-35B-A3B 在 SWE-bench Verified 上的表现。总体通过率从 19.5% 提升至 41.5%。面板 (a) 展示了演变轨迹，面板 (b) 展示了保留的补丁验证器和提交前检查，这些检查要求检查当前差异并执行定向测试。

Failed trace under the initial harness	Successful trace under the final SWE harness
E168 Make 'Window' inherit 'SQLiteNumericMixin'; bypass the cast only at the outer window node. E173 An ad-hoc import check stops at a missing 'asgiref' dependency. E248 Read 'tests/runtests.py', but never execute the relevant window-function suite. # the benchmark-relevant test is never executed E423 Inspect a nonempty source diff in the nested Django repository. E533 Submit the patch without a successful targeted test. # no test evidence gates submission END Official verifier: F2P 0/1; P2P 46/46. SQLite still errors near 'OVER'. # the submitted SQL rewrite remains invalid	E038 Guard 'SQLiteNumericMixin': skip the NUMERIC cast for window-compatible expressions. # place the fix where the inner window function is compiled E048 Make 'Window' use the mixin and mark it as window-compatible. E123 A local SQL check confirms that 'LAG(...) OVER (...)' is no longer wrapped by an invalid cast. E168 Run 'python tests/runtests.py expressions_window.tests': 49 tests OK (3 skipped). # targeted official suite passes before submission E183 Inspect the final source-only diff after the targeted suite passes. # inspect the current diff E198 Submit the verified patch. END Official verifier: F2P 1/1; P2P 46/46. Resolved. # all required tests resolve

E = archived trace event Steps are condensed; verifier counts are exact.

图 14 | Qwen3.5-35B-A3B 在 SWE-bench Verified 中 Django 窗口函数的执行轨迹。左侧：初始测试框架在未运行相关测试套件的情况下提交，未能通过唯一的失败转通过测试。右侧：保留的测试框架在提交前运行了该测试套件，并通过了所有必需的测试。



(a) Evolution trajectory.

GLM SWE: dependency-aware verification instruction

```

1 def build_verification_instruction() -> str:
2     return (
3         "Run the FAIL_TO_PASS test specified in the problem statement. "
4         "If the test fails, parse the failure output, then debug and fix it. "
5         "Repeat until the specific FAIL_TO_PASS test passes. "
6         "Do not submit while the FAIL_TO_PASS test is still failing."
7     )
8
9 + return (
10 +     "Run the FAIL_TO_PASS test specified in the problem statement. "
11 +     "Execute the exact test command. "
12 +     "If the test fails, parse the specific error. "
13 +     "If the failure is a ModuleNotFoundError, ImportError, "
14 +     "or missing dependency error, install the missing package "
15 +     "using 'pip install <package_name>' and then rerun the test. "
16 +     "For missing Python modules, identify the correct package name. "
17 +     "After resolving dependency issues, verify the test runs. "
18 +     "If the test still fails, debug and fix the code issue. "
19 +     "Do not submit while the FAIL_TO_PASS test is still failing "
20 +     "or cannot run due to missing dependencies."
21 + )

```

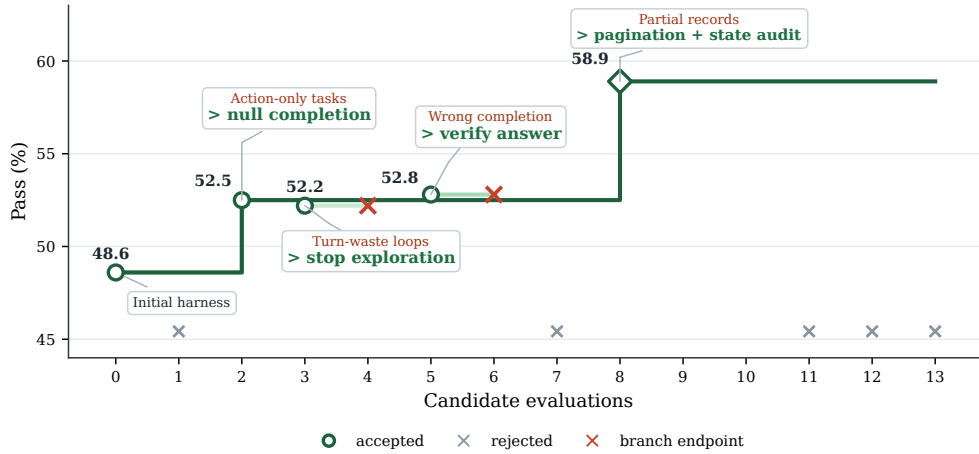
(b) Retained code-level edits.

图 15 | GLM-5 在 SWE-bench Verified 上的表现。总体通过率从 52.0% 提升至 55.5%。面板 (a) 展示了演变轨迹，面板 (b) 展示了保留的依赖感知验证循环，该循环在重新运行相关测试之前从导入失败中恢复。

Failed trace under the initial harness	Successful trace under the edited harness
<pre> T750 ModuleNotFoundError: No module named 'astropy' T999 ERROR: No matching distribution found for pyerfa T1052 Successfully installed pyerfa-2.0.1.5 T1091 ImportError: cannot import name '_compiler' from 'astropy.utils' T1115 ModuleNotFoundError: No module named 'extension_helpers' T1142 Successfully installed extension_helpers-1.4.0 setuptools-82.0.1 T1165 ModuleNotFoundError: No module named 'setuptools.dep_util' T1283 error: metadata-generation-failed T2291 ImportError: You appear to be trying to import astropy from an editable installation without building the extension modules first. # dependency repair does not converge T8098 Testing syntax of new file... Syntax check passed! # only a syntax-level check before submission. T8358 Please review your changes before final submission. T8395 Submitted. END Verifier: F2P 0/4; P2P 63/68. Patch unresolved. # submits with required tests unresolved </pre>	<pre> T2228 cd /workspace && python -c "from astropy.coordinates import ITRS, AltAz, HADEC; print('Imports successful!)" T2232 ModuleNotFoundError: No module named 'erfa' T2248 pip install pyerfa # missing dependency explicitly triggers installation T2361 ProxyError: Cannot connect to proxy. ERROR: No matching distribution found for pyerfa T2377 unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY && pip install pyerfa T2528 env grep -i proxy T2549 export http_proxy="" && export https_proxy="" && pip install pyerfa --trusted-host pypi.org # continues recovery after repeated proxy failures T2600 cd /workspace && pip install -e . --no-build-isolation T2604 ModuleNotFoundError: No module named 'extension_helpers' T2692 conda install -y pyerfa -c conda-forge # tries a fallback instead of abandoning verification T2941 cd /workspace && git diff --stat # reviews the final artifact before submission T2990 Please review your changes before final submission. T3121 Submitted. END Verifier: F2P 4/4; P2P 68/68. All required tests resolved. # external verifier resolves every required test </pre>

S = agent step T = tool call/result

图 16 | SWE-bench Verified Astropy 地心真赤道参考系 (ITRS) 变换轨迹, 针对 GLM-5。左图: 初始测试框架仅执行语法检查, 导致所有四个失败转通过 (FAIL-TO-PASS) 测试和五个通过转通过 (PASS-TO-PASS) 测试均未通过。右图: 提升后的测试框架修复了缺失依赖, 完成可执行验证, 并通过了全部四个失败转通过测试和全部 68 个通过转通过测试。



(a) Evolution trajectory.

MiniMax M2.5 AppWorld: completion-contract state auditor

```

1 def build_subagents() -> list[dict[str, str]]:
2     - return []
3     + return [{
4         "name": "state_auditor",
5         "description": "Verify task completion requirements before complete_task.",
6         "system_prompt": (
7             "If the task asks for information, pass answer with the extracted value. "
8             "If the task only requires an action, pass answer=None or omit answer. "
9             "Use available app APIs to confirm the action before completing."
10        ),
11    }]

```

MiniMax M2.5 AppWorld: pagination discipline prompt appendix

```

1 -react_prompt_appendix = ""
2 +react_prompt_appendix = ""
3 +Additional AppWorld strategy guidance for this run:
4 +
5 +When using paginated APIs, always check if more pages exist. After retrieving
6 +results, verify you have the complete dataset before filtering or acting on
7 +the data. If an API supports page_index or page_limit, determine whether you
8 +need to iterate through multiple pages to capture all relevant records.
9 +For temporal queries, use proper time boundaries.
10 +

```

(b) Retained code-level edits.

图 17 | MiniMax M2.5 在 AppWorld 上的表现。总体通过率从 48.6% 提升至 58.9%。面板 (a) 展示演化轨迹, 面板 (b) 展示保留的状态审计、分页和时间边界机制, 用于识别完整的目标记录集。

Failed trace under the initial AppWorld harness

```

T01 I need to find the note with individual shares and check Venmo payments.
T03 'search_notes(query='dinner')' returns 401 unauthorized.
T06 logs into simple_note with 'password=<REDACTED>'.
T10 retrieves note: Kayla $20, Sandra $19, Stephen $20, Jamie $15, Anita $23,
    Travis $39, Sabrina $39.
T12 logs into Venmo with 'password=<REDACTED>'.
T14 checks sent payment requests.
T22 checks received payment requests.
T25 'show_transactions(access_token=<REDACTED>, page_index=0, page_limit=20)'.
T26 checks page 1.
T27 checks page 2.
T28 checks page 3.
T29 checks page 4 without finding the relevant paid dinner set.
T30 'none of the dinner participants have paid their shares yet'; decides to
    request all seven.
    # incomplete pagination leads to a wrong payment conclusion
T32 creates requests for Kayla, Sandra, Stephen, Jamie, Anita, Travis, and
    Sabrina.
    # over-requests all 7 users
T39 'apis.supervisor.complete_task()'.
    # finalizes the wrong action set
END Evaluation fails.

```

S = agent step T = tool call/result

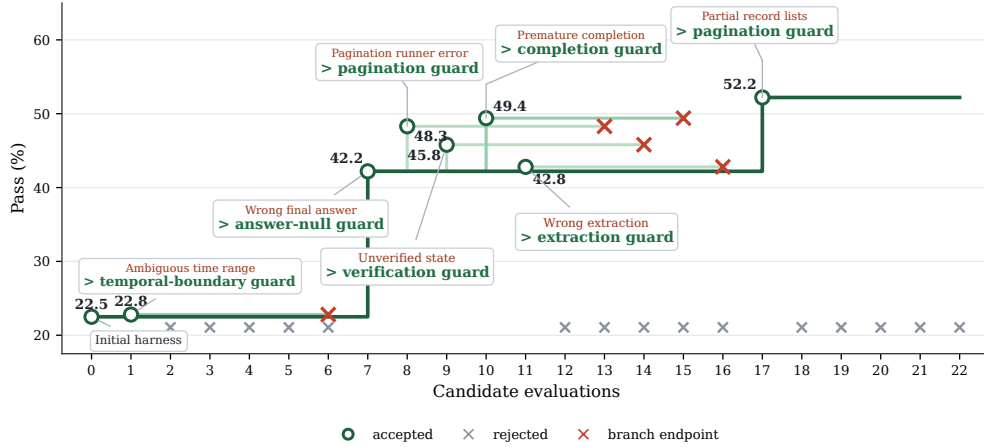
Successful trace under the final AppWorld harness

```

T01 I need to find the note with individual shares from the dinner.
T03 'search_notes(query='dinner')' returns 401 unauthorized.
T07 logs into simple_note with 'password=<REDACTED>'.
T11 retrieves the same seven-person share list.
T13 logs into Venmo; output access token redacted.
T15 checks sent payment requests.
T16 checks received payment requests.
T19 'Let me use pagination.' Calls 'show_transactions(..., page_index=1,
    page_limit=20)'.
    # follows pagination discipline from the harness edit
T20 notes temporal boundary: dinner was yesterday, 2023-05-19; checks page 2.
    # applies the relevant temporal boundary
T21 continues transaction pagination through page 3.
T22 continues transaction pagination through page 4.
T23 continues transaction pagination through page 5.
T24 continues transaction pagination through page 6.
T25 already paid: Sabrina $39, Kayla $20, Anita $23, Jamie $15; remaining Sandra
    $19, Stephen $20, Travis $39.
    # verifies complete data before acting
T26 creates exactly three payment requests for Sandra, Stephen, and Travis.
    # requests only the remaining shares
T27 'apis.supervisor.complete_task()'.
END Evaluation passes.

```

图 18 | AppWorld Venmo 轨迹，针对 MiniMax M2.5。左图：初始测试框架向全部七名参与者请求付款，未考虑先前交易。右图：提升后的测试框架遍历所有交易页面，识别出已付款的四名参与者，仅向剩余三名请求付款。



(a) Evolution trajectory.

Qwen3.5 AppWorld: completion-contract prompt guard

```

1 1 def build_react_prompt_template() -> str:
2   prompt = BASE_REACT_PROMPT
3   prompt += """
4 +Completion Contract Check:
5 +Before calling complete_task(), decide whether the task asks for an answer.
6 +For action-only tasks, pass answer=None or omit answer.
7 +For information tasks, pass the exact extracted entity, number, or string.
8 +Do not send a summary sentence unless the task explicitly requests one.
9 +"""
3 10 return prompt

```

Qwen3.5 AppWorld: pagination prompt guard

```

1 1 def build_react_prompt_template() -> str:
2   prompt = BASE_REACT_PROMPT
3   prompt += """
4 +Critical State Retrieval Rule: Pagination Check.
5 +Before filtering, deleting, or mutating list records, inspect pagination fields.
6 +If page_index, page_limit, total_count, or has_more appears, iterate all pages.
7 +Build decisions from the complete retrieved record set, not from the first page.
8 +Re-check the target object after pagination before calling complete_task().
9 +"""
3 10 return prompt

```

(b) Retained code-level edits.

图 19 | Qwen3.5-35B-A3B 在 AppWorld 上的表现。总体通过率从 22.5% 提升至 52.2%。面板 (a) 展示演化轨迹，面板 (b) 展示保留的完成契约和分页保护机制，要求完整检索并区分仅动作任务与信息请求。

Failed trace under the initial harness

```

S01 I need to understand the Spotify APIs for the song library and play counts.
T02 'show_song_library' exposes 'page_index' and 'page_limit', default limit 5.
S03 The API requires an access_token. I need to get the Spotify password first;
    try login with email/password variants.
T04 Login succeeds with the full email. Fetches the song library and reports
    'Total songs fetched: 19'.
    # acts on an incomplete retrieved set
END Evaluation fails: predicted answer is 'null', expected 'distant love'.
    # wrong completion answer

```

S = agent step T = tool call/result

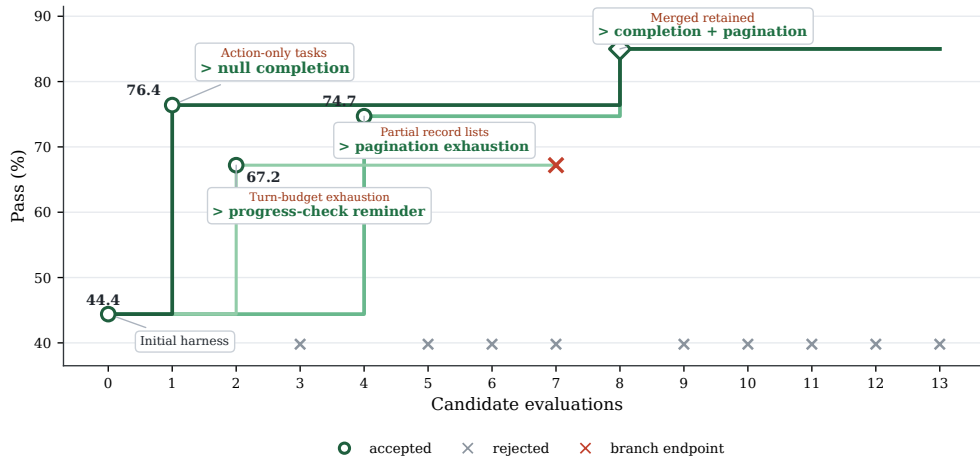
Successful trace under the edited harness

```

S01 I need to understand the Spotify APIs to list songs and their play counts.
T02 'show_song_library' shows pagination fields: 'page_index', 'page_limit'.
S03 'show_song_library' lacks play counts; check 'show_song_privates'.
S04 'show_song_privates' does not expose play counts; check 'show_song'.
    # keeps inspecting before answering
T04 'show_song' response includes 'play_count'.
S08 The API supports pagination, so I will start with page 0 and iterate through
    all pages to ensure I get all songs.
    # follows pagination guard from harness edit
T08 Fetch page 0 with 'page_limit=20'; if full, fetch page 1; collect all songs;
    then call 'show_song' for each song and choose the highest play count.
END Evaluation passes: 2/2 tests passed, answers match.
    # completion contract now matches evaluator

```

图 20 | AppWorld Spotify 轨迹，针对 Qwen3.5-35B-A3B。左图：初始测试框架在检索不完整后返回空播放次数结果。右图：提升后的测试框架遍历库页面，比较检索到的播放次数，并返回预期歌曲。



(a) Evolution trajectory.

GLM AppWorld: merged ReAct prompt appendices

```

1 1 def build_react_prompt_template() -> str:
2 2     prompt = BASE_REACT_PROMPT
3 3     prompt += """
4 4 +Important completion rule:
5 5 +If the task asks you to perform actions
6 6 +(create, update, delete, send, follow, like, etc.)
7 7 +without asking a question, call
8 8 +apis.supervisor.complete_task() with no answer argument
9 9 +or answer=None. Do not provide a summary of actions taken.
10 10 +Only pass an answer when the task explicitly asks for information.
11 11 +
12 12 +For APIs that return paginated results
13 13 +(with page_index and page_limit parameters),
14 14 +retrieve ALL pages before processing or mutating state.
15 15 +Increment page_index starting from 0 until an empty list is returned.
16 16 +Do not assume the first page contains all results.
17 17 +Before bulk actions, verify that pagination is exhausted.
18 18 +"""
19 19     return prompt

```

(b) Retained code-level edits.

图 21 | GLM-5 在 AppWorld 上的表现。总体通过率从 44.4% 提升至 85.0%。面板 (a) 展示演化轨迹，面板 (b) 展示保留的动作与信息完成规则以及在状态变更前应用的穷举分页要求。

Failed trace under the initial harness

```
S01 Inspect supervisor and Venmo APIs, then login to Venmo.
T03 show_api_doc(venmo, show_transactions) exposes page_index and page_limit.
# pagination fields visible but unused
T10 show_transactions(... direction="received") confirms recent Venmo activity.
T12 show_transactions(..., page_limit=20) returns only page 0.
# no page_index=1 follow-up
S13 Build friend transactions from the first retrieved page.
T14 Create comments and likes for ids 8280..8288 and 8295 only.
# misses required ids 8297 and 8298
END complete_task(answer="Added ... 10 transactions ..."). Evaluation fails.
# non-null summary for action-only task
```

S = agent step T = tool call/result

Successful trace under the merged harness

```
S01 Inspect supervisor and Venmo APIs, then login to Venmo.
T03 show_api_doc(venmo, show_transactions) exposes page_index and page_limit.
T12 show_transactions(..., page_index=0, page_limit=20) returns the first page.
T14 show_transactions(..., page_index=1, page_limit=20) returns ids 8297 and 8298.
# follows pagination exhaustion prompt
T15 show_transactions(..., page_index=2, page_limit=20) returns [].
# verifies empty final page
S19 Use all 12 transaction ids: 8280..8288, 8295, 8297, 8298.
T19 Create comments and likes for all 12 exhausted-page transaction ids.
# mutates complete_retrieved_set
END complete_task(answer=None). Evaluation passes.
# follows action-only completion contract
```

图 22 | AppWorld Venmo 轨迹，针对 GLM-5。左图：初始测试框架仅读取第一个交易页面并提供不必要的文本答案。右图：提升后的测试框架遍历所有页面，变更全部 12 条目标记录，并以空答案完成仅动作任务。